



Mercantec

April-Maj 2026

Produkt Rapport – H5 – Mini

Datatekniker med speciale i programmering



Lavet af

Alexander Baravitski

Oliver Lund Holm G. S.



Mercantec

Indholdsfortegnelse

INDLEDNING	3
<i>Baggrund.....</i>	3
<i>Problemstilling.....</i>	3
<i>Afgrænsning.....</i>	4
PRODUKTBESKRIVELSE.....	5
<i>Formål & målgruppe</i>	5
<i>Overordnet funktionalitet.....</i>	5
<i>Teknologi</i>	9
<i>Arkitektur.....</i>	10
KRAV & MÅL	11
<i>Faglige mål</i>	11
<i>Ikke funktionelle-krav</i>	12
<i>Test på krav.....</i>	12
PLANLÆGNING	13
<i>Tidsplan</i>	13
<i>Milepæle</i>	14
<i>Risiko & mitigering.....</i>	14
<i>Samarbejde & kommunikation.....</i>	15
<i>Værktøjer til planlægning.....</i>	15
UDFØRELSE	16
<i>Arbejdsproces</i>	16
<i>Centrale tekniske valg</i>	16

<i>Implementering</i>	19
<i>Server</i>	19
<i>Editor</i>	22
<i>Invite/Accept</i>	23
<i>Database</i>	25
<i>Kvalitetssikring undervejs</i>	25
<i>Udfordringer under udvikling</i>	26
RESULTAT & EVALUERING	27
<i>Test</i>	27
<i>Automatiserede tests</i>	27
<i>Manuelle tests</i>	27
<i>Krav vs. leveret</i>	28
<i>Kendte begrænsninger</i>	29
REFLEKSION	30
<i>Egen læring</i>	30
<i>Fremtidige forbedringer</i>	30
<i>Afsluttende kommentarer</i>	31

INDLEDNING

Baggrund

Projektet er udviklet på baggrund af en opgave stilet af en uddannelsesinstitution, som har udtrykt behov for en digital platform til at understøtte studerendes arbejde med noter, samarbejde og organisering af fagligt indhold.

Selvom der allerede findes værktøjer som Google Docs og OneNote, vurderede institutionen, at disse ikke i tilstrækkelig grad understøtter behovet for både struktureret organisering og samarbejde i én samlet løsning.

Problemstilling

Formålet med dette projekt er at udvikle en webbaseret platform, der giver studerende mulighed for at oprette, organisere og samarbejde om noter på en struktureret måde.

Målgruppen for systemet er studerende, som har behov for at arbejde med noter både individuelt og i grupper. Problemstillingen opstår i forbindelse med, at eksisterende værktøjer enten fokuserer på dokumentredigering eller organisering, men sjældent understøtter begge dele effektivt i én samlet løsning. Dette gør det vanskeligt for brugere at opretholde en struktureret tilgang til deres noter, samtidig med at de samarbejder med andre.

Projektet undersøger derfor, hvordan man kan udvikle en løsning, hvor noter organiseres i fag og emner, samtidig med at flere brugere kan redigere den samme note i realtid. En central udfordring er at håndtere samtidige ændringer fra flere brugere uden at skabe datakonflikter, samt at sikre, at dokumentets tilstand forbliver konsistent på tværs af klienter.

Derudover arbejdes der med, hvordan dokumentdata kan persisteres effektivt i en database, uden at gå på kompromis med performance, særligt i forbindelse med hyppige opdateringer fra real-time samarbejde.

Projektet har således til formål at kombinere struktureret organisering af noter med real-time samarbejde i én samlet platform, der er målrettet studerendes arbejdsproces.

Afgrænsning

Projektet er afgrænset til udviklingen af en webbaseret samarbejdsplatform til noter med fokus på real-time redigering og deling mellem brugere. Der er primært fokuseret på kernefunktioner som autentifikation, organisering af noter, collaboration via Yjs/Hocuspocus samt invitation og adgangsstyring mellem brugere.

Funktioner som avanceret tekstformatering, offline-support, versionshistorik, mobilapp, og e-mail invitationer er ikke implementeret i denne version. Der er desuden lagt vægt på funktionalitet og systemarkitektur frem for fuld UI/UX optimering.

PRODUKTBESKRIVELSE

Formål & målgruppe

Formålet med produktet er at udvikle en webbaseret platform, der gør det muligt for brugere at oprette, organisere og samarbejde om noter på en struktureret måde. Systemet kombinerer organisering af noter gennem fag og emner med real-time redigering, hvor flere brugere kan arbejde i samme dokument samtidig.

Målgruppen for systemet er primært studerende, som har behov for at strukturere deres noter og samarbejde med andre i forbindelse med undervisning og gruppearbejde. Derudover kan platformen også anvendes af mindre teams, der har behov for en enkel løsning til vidensdeling og samarbejde.

Overordnet funktionalitet

Systemet består af en række centrale funktioner, der tilsammen understøtter oprettelse, organisering og samarbejde om noter.

Oprettelse og organisering af noter

Brugeren kan oprette noter og organisere dem i en struktur bestående af fag og emner. Dette gør det muligt at opbygge en logisk og overskuelig struktur, hvor noter er knyttet til specifikke fagområder.

Real-time redigering

Systemet understøtter real-time samarbejde, hvor flere brugere kan redigere den samme note samtidig. Ændringer synkroniseres løbende mellem klienter, så alle brugere ser opdateringer med det samme.

Automatisk lagring af noter

Noter gemmes automatisk i databasen, hvilket sikrer, at brugerens arbejde ikke går tabt. Systemet anvender en mekanisme til at reducere antallet af databasekald, samtidig med at ændringer gemmes løbende.

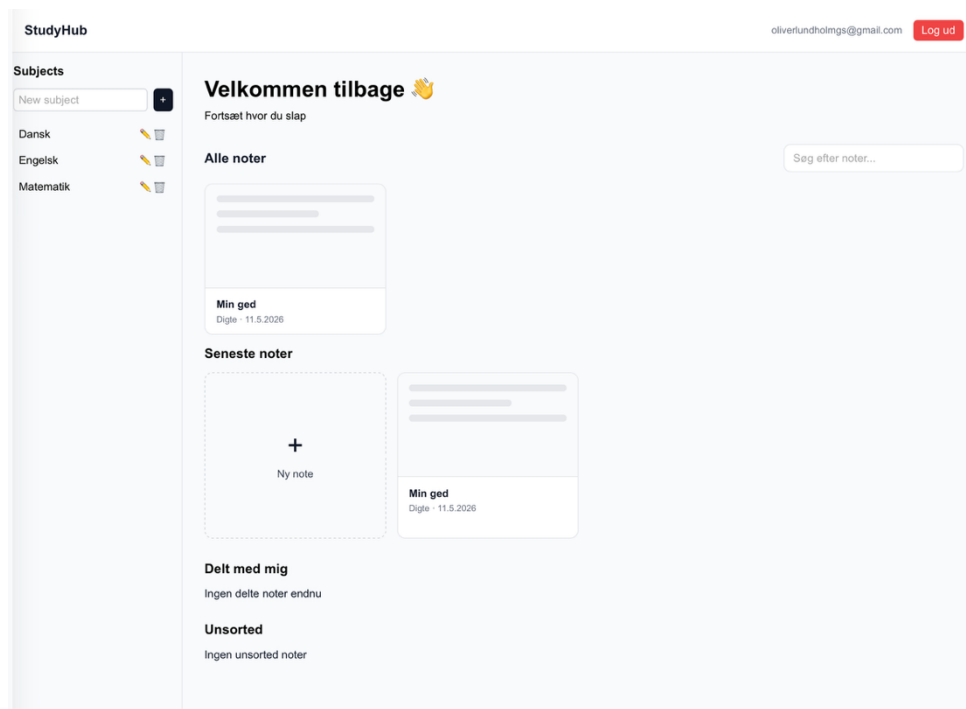
Brugerhåndtering

Brugere kan logge ind og få adgang til deres egne noter. Systemet sikrer, at brugere kun har adgang til relevante data.

Navigation og brugergrænseflade

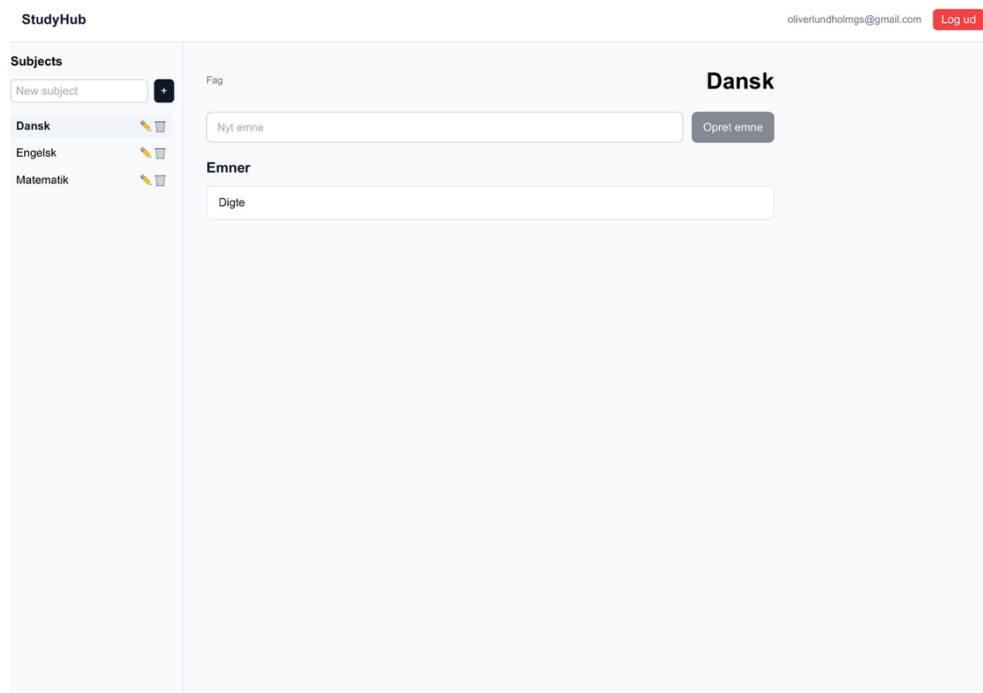
Brugeren kan navigere mellem fag, emner og noter via en sidebar, hvilket giver et hurtigt overblik over eksisterende indhold og gør det nemt at tilgå specifikke noter.

Figur 1 viser systemets forside og overordnede navigation.



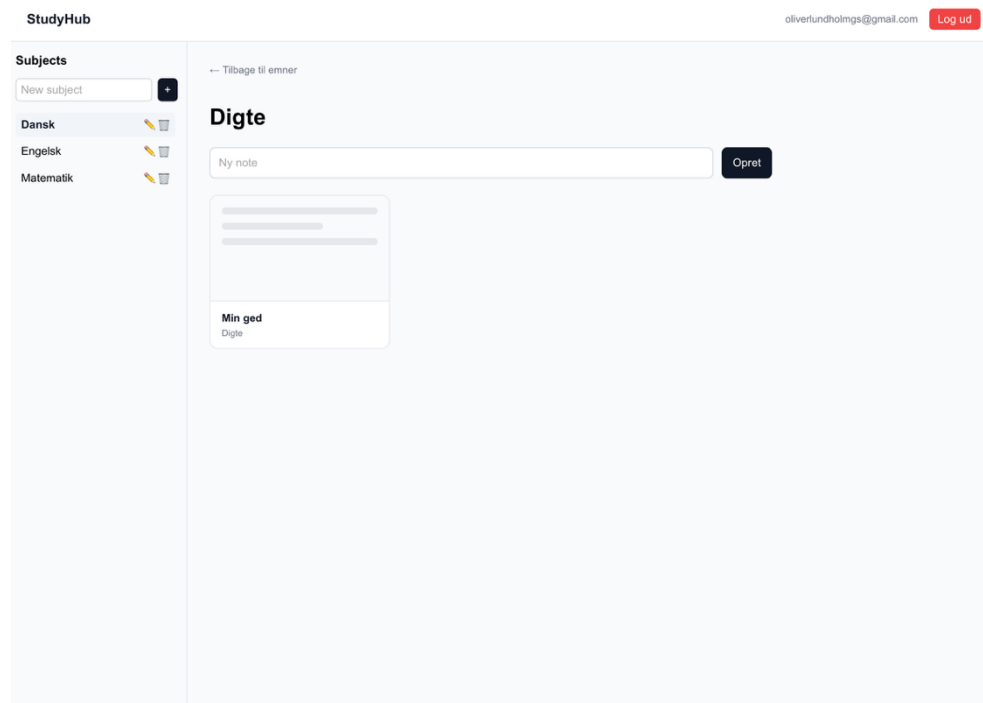
(Figur 1)

Figur 2 viser organisering af noter gennem fagstruktur.



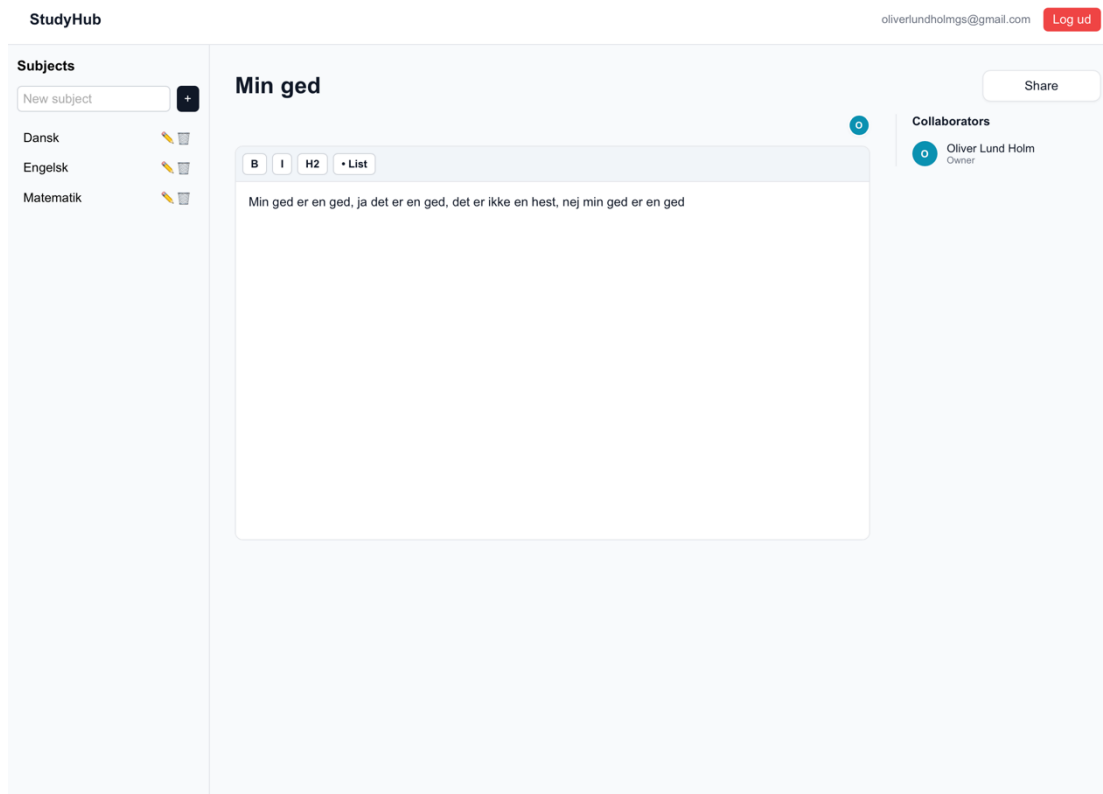
(Figur 2)

Figur 2-3 viser opdeling af noter i emner under et fag.



(Figur 3)

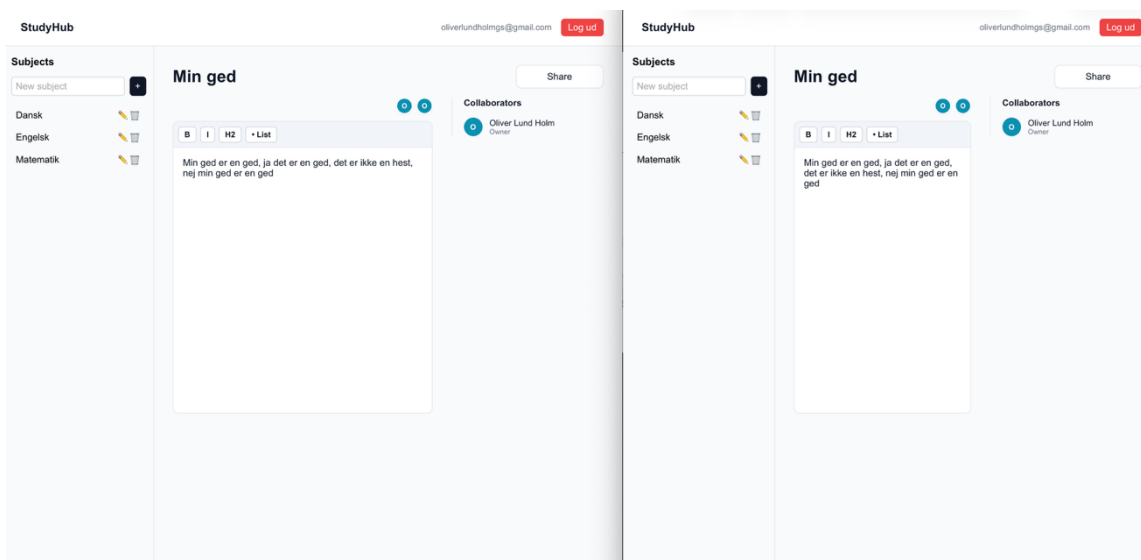
Figur 4 viser rich-text editoren anvendt til redigering af noter.



(Figur 4)

Figur 5 viser real-time samarbejde, hvor flere brugere kan redigere samme note samtidigt.

(Dog den samme bruger i screenshottet)



(Figur 5)

Teknologi

FRONTEND

Next.js (React) anvendes til at opbygge brugergrænsefladen og håndtere routing samt server-side rendering. Tiptap bruges som rich-text editor, da den er fleksibel og kan udvides med real-time funktionalitet.

BACKEND

Next.js API routes håndterer serverlogik og kommunikation med databasen. Drizzle ORM anvendes til type-sikker databaseadgang, og PostgreSQL bruges som relationsdatabase til lagring af brugere, noter og struktur.

REALTIME/COLLABORATION

Yjs anvendes til at håndtere real-time dokumentdata og konflikthåndtering via CRDT. Hocuspocus fungerer som WebSocket-server og håndterer synkronisering mellem klienter.

DRIFT/DEPLOY

Systemet er udviklet lokalt, men kan senere udvides med containerisering (Docker) for nem deployment og skalering.

Arkitektur

(Bruger / Client)



(Next.js Frontend)



(API Routes / Server Actions)



(Database (PostgreSQL))



(Hocuspocus WebSocket server)



(Yjs CDRT Sync (real-time sync))

KRAV & MÅL

Faglige mål

Krav / mål	Hvordan er det opfyldt?	Reference
Brugeren skal kunne oprette og organisere noter	Systemet understøtter oprettelse af fag, emner og noter via frontend og database	Produktbeskrivelse / bilag
Systemet skal understøtte real-time samarbejde	Flere brugere kan redigere samme note samtidigt via Yjs og Hocuspocus	Realtidsfunktionalitet
Data skal persisteres i database	Noter gemmes i PostgreSQL via Drizzle ORM	Arkitektur / backend
Brugeren skal kunne logge ind	NextAuth anvendes til autentifikation	Backend
Systemet skal understøtte strukturering af indhold	Noter er organiseret gennem subjects og topics	Database diagram

Ikke funktionelle-krav

Krav	Status	Kommentar
Real-time opdateringer skal ske uden mærkbar forsinkelse	Opfyldt	Ændringer synkroniseres øjeblikkeligt via WebSockets
Systemet skal kunne håndtere samtidige ændringer	Opfyldt	Yjs anvender CRDT til konflikthåndtering
Data skal gemmes automatisk	Opfyldt	Dokumenter gemmes løbende via debounced persistence
Systemet skal kunne vedligeholdes	Delvist opfyldt	Projektet er opdelt i komponenter og services
Systemet skal være responsivt	Delvist opfyldt	Grundlæggende responsive layouts er implementeret

Test på krav

For at verificere systemets funktionalitet er der udført en række tests af de centraler funktioner. Der er blandt andet testet, at brugere kan oprette fag, emner og noter, samt at noter gemmes korrekt i databasen.

Derudover er real-time samarbejde testet ved at åbne samme note i flere browserfaner og verificere, at ændringer synkroniseres øjeblikkeligt mellem klienterne. Systemet er også testet ved genindlæsning af siden for at sikre, at dokumenttilstanden persisteres korrekt.

PLANLÆGNING

Tidsplan

Fase	Aktivitet	Start	Slut	Status
1	Analyse og idéudvikling	Uge 1	Uge 1	Opfyldt
2	Opsætning af Next.js og database	Uge 1	Uge 2	Opfyldt
3	Implementering af struktur (subjects/topics/notes)	Uge 2	Uge 3	Opfyldt
4	Implementering af autentifikation	Uge 3	Uge 3	Opfyldt
5	Real-time samarbejde med Yjs og Hocuspocus	Uge 4	Uge 5	Opfyldt
6	Persistence og lagring af dokumenter	Uge 5	Uge 5	Opfyldt
7	UI forbedringer og testing	Uge 5	Uge 5	Igangværende

Milepæle

1. Grundlæggende noteplatform implementeret
2. Database og autentifikation opsat
3. Real-time samarbejde fungerende
4. Persistence af dokumenttilstand implementeret
5. Endelig testing og rapportskrivning påbegyndt

Risiko & mitigering

Risiko	Sandsynlighed	Konsekvens	Tiltag
Problemer med real-time synkronisering	H	H	Undersøgelse af Yjs og Hocuspocus dokumentation
Datatab ved samtidige ændringer	M	H	Implementering af persistence og automatisk lagring
Tidsmangel	M	M	Fokus på kernefunktionalitet frem for avancerede features
Kompleksitet i WebSocket-kommunikation	M	H	Anvendelse af abstraherede biblioteker som Hocuspocus

Samarbejde & kommunikation

Projektet er udviklet som et samarbejde mellem gruppens medlemmer, og arbejdet er løbende blevet organiseret gennem Git versionsstyring. Derudover er der anvendt commits til at dokumentere udviklingsprocessen og sikre versionskontrol under implementeringen. Microsoft live share extension til vscode er blevet anvendt i store dele af projektet.

Vejledning og teknisk sparring er løbende blevet anvendt til at afklare tekniske problemstillinger.

Værktøjer til planlægning

Projektet er primært blevet planlagt gennem en iterativ udviklingsproces, hvor funktionalitet er blevet implementeret og forbedret løbende. Git og GitHub er anvendt til versionsstyring og dokumentation af ændringer i projektet.

Derudover er opgaver og idéer løbende blevet struktureret gennem egne noter og prioritering af kernefunktionalitet.

UDFØRELSE

Arbejdsproces

Projektet er udviklet gennem en iterativ arbejdsproces, hvor funktionalitet løbende er blevet implementeret, testet og forbedret. Udviklingen startede med grundlæggende strukturering af data og brugergrænseflade, hvorefter mere avancerede funktioner som real-time samarbejde blev implementeret.

Der er arbejdet med versionsstyring gennem Git, hvor ændringer løbende er blevet opdelt i commits for at dokumentere udviklingsprocessen. Funktionalitet er typisk blevet udviklet i mindre trin, så nye features kunne testes løbende under implementeringen.

Fokus under udviklingen har været at prioritere kernefunktionalitet først og derefter forbedre arkitektur og brugeroplevelse gradvist.

Centrale tekniske valg

VALG 1 – TIPTAP

- **Beslutning:**

Tiptap blev valgt som rich-text editor til systemet

- **Alternativer:**

Der blev overvejet at udvikle en custom editor fra bunden eller anvende enklere editors som textarea-baserede løsninger.

- **Konsekvens:**

Tiptap gjorde det muligt hurtigt at implementere avanceret editor-funktionalitet og understøtte integration med real-time samarbejde via Yjs. Samtidig reducerede det kompleksiteten sammenlignet med at udvikle en editor fra bunden.

VALG 2 – HOCUSPOCUS + YJS

- **Beslutning:**

Yjs og Hocuspocus blev valgt til håndtering af real-time samarbejde.

- **Alternativer:**

Der blev overvejet at udvikle en custom WebSocket-løsning uden abstraherede biblioteker.

- **Konsekvens:**

Ved at anvende Yjs og Hocuspocus kunne systemet håndtere samtidige ændringer og konflikthåndtering gennem CRDT-teknologi. Dette reducerede udviklingstiden betydeligt og gjorde løsningen mere stabil sammenlignet med en custom implementering.

VALG 3 – NEXT.JS

- **Beslutning:**

Next.js blev valgt som framework til både frontend og backend funktionalitet.

- **Alternativer:**

Der blev overvejet at anvende React med en separat backend, eksempelvis Express eller NestJS.

- **Konsekvens:**

Ved at anvende Next.js kunne frontend, API routes og routing samles i ét framework, hvilket reducerede kompleksiteten og gjorde udviklingsprocessen mere effektiv. Samtidig gjorde server components det muligt at hente data direkte fra databasen uden unødvendige API-kald.

VALG 4 – POSTGRESQL

- **Beslutning:**

PostgreSQL blev valgt som relationsdatabase til lagring af systemets data.

- **Alternativer:**

Der blev overvejet dokumentbaserede databaser som MongoDB.

- **Konsekvens:**

PostgreSQL passede godt til projektets relationelle datastruktur, hvor brugere, fag, emner og noter har tydelige relationer til hinanden. Databasen gav samtidig mulighed for stabile relationer, constraints og struktureret datahåndtering.

VALG 5 – DRIZZLE ORM

- **Beslutning:**

Drizzle ORM blev valgt til databaseadgang og håndtering af migrationer.

- **Alternativer:**

Der blev overvejet Prisma samt raw SQL queries.

- **Konsekvens:**

Drizzle gav type-sikker adgang til databasen og passede godt sammen med TypeScript og monorepo-strukturen. Samtidig gav det mere direkte kontrol over SQL-strukturen sammenlignet med mere abstraherede ORM-løsninger.

Implementering

Server

Funktionen gemmer den aktuelle Yjs-dokumenttilstand i databasen, så noter kan gendannes og synkroniseres mellem brugere.

```
async function saveDocument(documentName: string, document: Y.Doc)
  const noteId = documentName.replace("note-", "");

  const update = Y.encodeStateAsUpdate(document);
  const base64State = Buffer.from(update).toString("base64");

  await db
    .update(notes)
    .set({
      ydocState: base64State,
      updatedAt: new Date(),
    })
    .where(eq(notes.id, noteId));

  console.log("📄 Saved document:", documentName);
}
```

onLoadDocument bliver kaldt, når en bruger åbner en note. Funktionen henter den gemte Yjs-tilstand fra databasen og anvender den på dokumentet, så brugeren ser den seneste version af noten.

onChange bliver kaldt ved ændringer i dokumentet. Funktionen anvender en debounce-mekanisme, så dokumentet ikke gemmes ved hvert tastetryk, men først efter en kort periode uden ændringer.

onStoreDocument bruges som en ekstra sikkerhedsmekanisme til at gemme dokumentets tilstand direkte, når Hocuspocus-serveren udfører en lagringsoperation.

```
async onLoadDocument({ documentName, document }) {
  const noteId = documentName.replace("note-", "");

  const note = await db.query.notes.findFirst({
    where: eq(notes.id, noteId),
  });

  if (!note?.ydocState) {
    return document;
  }

  try {
    const update = Buffer.from(note.ydocState, "base64");
    Y.applyUpdate(document, update);

    console.log("📄 Loaded document:", documentName);
  } catch (error) {
    console.error("⚠️ Invalid Yjs state:", documentName, error);
  }

  return document;
},

async onChange({ documentName, document }) {
  scheduleSave(documentName, document);
},

async onStoreDocument({ documentName, document }) {
  await saveDocument(documentName, document);
},
```

Funktionen `scheduleSave` bruges til at planlægge lagring af dokumentet med `debounce`. Hvis brugeren fortsætter med at skrive, nulstilles timeren, så databasen ikke opdateres unødvendigt ofte. Dette reducerer antallet af databasekald og forbedrer performance.

Dette kodeuddrag viser, hvordan editoren kobles sammen med Yjs og Hocuspocus. Collaboration synkroniserer dokumentets indhold mellem brugere, mens `CollaborationCaret` viser andre brugeres markører og navne i realtid.

```
function scheduleSave(documentName: string, document: Y.Doc) {
  const existing = pendingSaves.get(documentName);

  if (existing?.timer) {
    clearTimeout(existing.timer);
  }

  const timer = setTimeout(async () => {
    try {
      await saveDocument(documentName, document);
    } catch (error) {
      console.error("❌ Failed to save document:", documentName, error);
    } finally {
      pendingSaves.delete(documentName);
    }
  }, SAVE_DEBOUNCE_MS);

  pendingSaves.set(documentName, {
    timer,
    document,
  });
}
```

Editor

Dette kodeuddrag viser, hvordan editoren kobles sammen med Yjs og Hocuspocus. Collaboration synkroniserer dokumentets indhold mellem brugere, mens CollaborationCaret viser andre brugeres markører og navne i realtid.

```
const editor = useEditor(  
  {  
    extensions: [  
      StarterKit.configure({  
        undoRedo: false,  
      }),  
      Collaboration.configure({  
        document: ydoc,  
        field: "default",  
      }),  
      CollaborationCaret.configure({  
        provider,  
        user,  
      }),  
    ],  
    immediatelyRender: false,  
  })
```

Her oprettes et Yjs-dokument og en Hocuspocus provider for den aktuelle note. Når brugeren forlader noten, lukkes forbindelsen og dokumentet ryddes op, så der ikke opstår unødvendige WebSocket-forbindelser eller memory leaks.

```
useEffect(() => {  
  const doc = createYDoc();  
  const hocuspocusProvider = createProvider(noteId, doc);  
  
  setYdoc(doc);  
  setProvider(hocuspocusProvider);  
  
  return () => {  
    hocuspocusProvider.destroy();  
    doc.destroy();  
    setYdoc(null);  
    setProvider(null);  
  };  
}, [noteId]);
```

Invite/Accept

Tjekker om den nuværende bruger har adgang til at oprette en invite

```
async function hasAccess(noteId: string, userId: string) {
  const note = await db.query.notes.findFirst({
    where: eq(notes.id, noteId),
  });

  if (!note) return false;

  if (note.ownerId === userId) {
    return true;
  }

  const collaborator = await db.query.noteCollaborators.findFirst({
    where: and(
      eq(noteCollaborators.noteId, noteId),
      eq(noteCollaborators.userId, userId),
    ),
  });

  return !!collaborator;
}
```

Her genereres en unik token, som bruges til at identificere invitationen. Token gemmes i note-Invites-tabellen sammen med noteld og den bruger, der oprettede invitationen. Til sidst returneres en URL, som kan deles med andre for at acceptere invitationen.

```
const token = crypto.randomBytes(32).toString("hex");

await db.insert(noteInvites).values({
  noteId,
  token,
  createdByUserId: session.user.id,
});

const inviteUrl = `${process.env.NEXT_PUBLIC_APP_URL}/invite/${token}`;
```


Koden implementerer et invitationssystem til deling af noter mellem brugere. Systemet validerer først, om invitationen og den tilhørende note eksisterer i databasen, hvorefter brugerens relation til noten kontrolleres. Hvis brugeren ikke allerede er ejer eller collaborator, oprettes der en ny collaborator-forbindelse med rollen som editor. Derudover oprettes en placement-reference, som sikrer at noten kan tilgås og organiseres korrekt i systemet. Løsningen anvender databasevalidering og betingede inserts for at undgå dubletter samt sikre dataintegritet

```
if (!invite) {
  return NextResponse.json(
    { error: "Invite not found" },
    { status: 404 },
  );
}

const note = await db.query.notes.findFirst({
  where: eq(notes.id, invite.noteId),
});

if (!note) {
  return NextResponse.json(
    { error: "Note not found" },
    { status: 404 },
  );
}

const userId = session.user.id;

if (note.ownerId === userId) {
  return NextResponse.json({
    noteId: note.id,
    message: "You already own this note",
  });
}

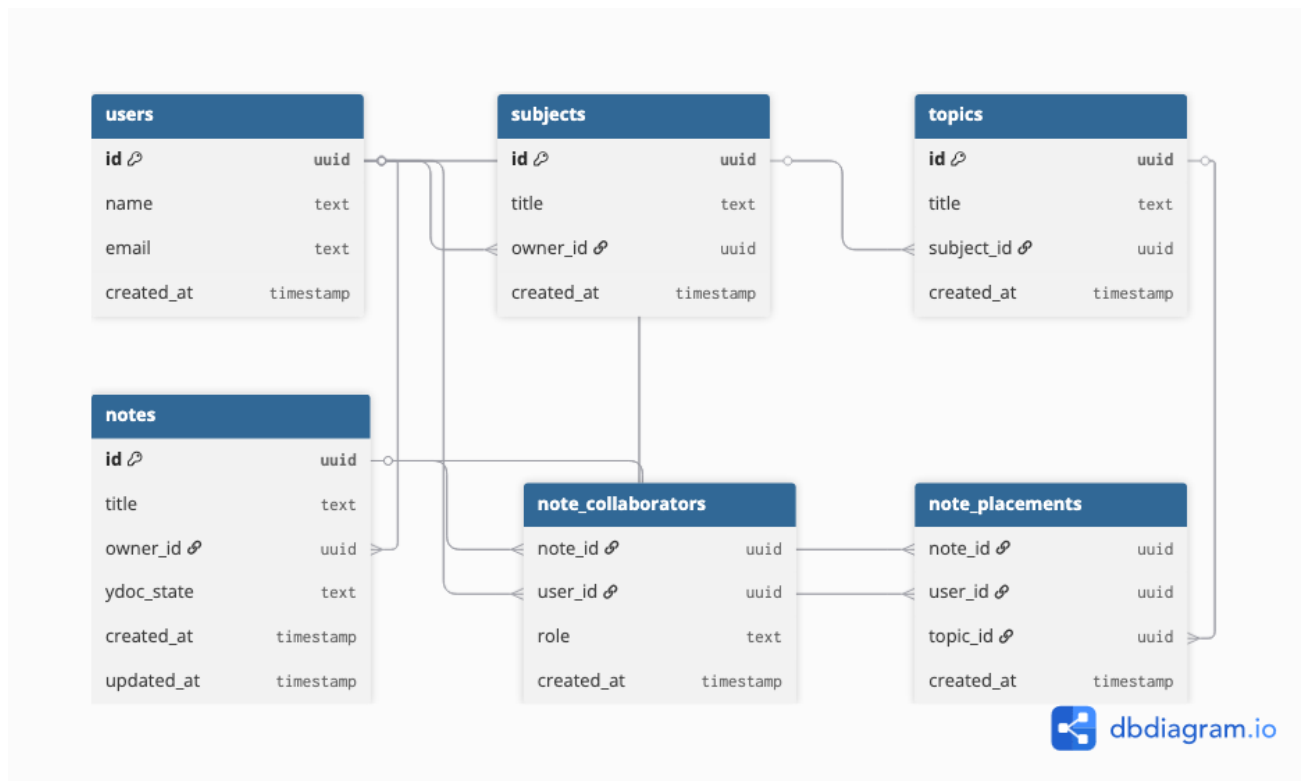
const existingCollaborator =
  await db.query.noteCollaborators.findFirst({
    where: and(
      eq(noteCollaborators.noteId, note.id),
      eq(noteCollaborators.userId, userId),
    ),
  });

if (!existingCollaborator) {
  await db.insert(noteCollaborators).values({
    noteId: note.id,
    userId,
    role: "editor",
  });
}

const existingPlacement =
  await db.query.notePlacements.findFirst({
    where: and(
      eq(notePlacements.noteId, note.id),
      eq(notePlacements.userId, userId),
    ),
  });

if (!existingPlacement) {
  await db.insert(notePlacements).values({
    noteId: note.id,
    userId,
    topicId: null,
  });
}
```

Database



Kvalitetssikring undervejs

Under udviklingen er der anvendt ESLint og formatering for at sikre en ensartet kodelstil og reducere simple fejl i koden.

Derudover er systemet løbende blevet testet manuelt under implementeringen. Særligt real-time funktionaliten er blevet testet gennem flere browserfaner og klienter for at verificere, at ændringer synkroniseres korrekt mellem brugere.

Persistence af dokumenter er ligeledes blevet testet ved genindlæsning af sider og genåbning af noter for at sikre, at dokumenttilstanden gemmes korrekt i databasen.

Udfordringer under udvikling

En af de største udfordringer under udviklingen var implementeringen af real-time samarbejde mellem flere brugere. Særligt håndtering af samtidige ændringer og persistence af dokumenttilstand krævede en del undersøgelser og eksperimentering.

Under udviklingen opstod der blandt andet problemer med synkronisering af editor-state mellem klienter, håndtering af WebSocket-forbindelser samt kompatibilitet mellem forskellige versioner af Tiptap extensions. Dette medførte fejl relateret til types og integration mellem collaboration extensions.

Derudover opstod der udfordringer i forbindelse med hydration og rendering i Next.js, særligt ved brug af client components og real-time editorfunktionalitet. Disse problemer blev løst ved at opdele ansvar mellem server components og client components samt ved at strukturere komponenterne mere modulært.

Persistence af Yjs dokumentdata var ligeledes en udfordring, da dokumenttilstanden skulle lagres effektivt uden at skrive til databasen på hvert tastetryk. Dette blev løst gennem en debounced persistence-strategi, hvor ændringer gemmes løbende med reduceret antal databasekald.

Projektet har generelt krævet løbende justeringer af arkitektur og struktur i takt med, at systemets kompleksitet voksede.

RESULTAT & EVALUERING

Test

Automatiserede tests

Automatiserede test er endnu ikke implementeret, da der primært har været fokus på manuel test og kernefunktionalitet.

Manuelle tests

Der blev under udviklingen lagt særligt fokus på manuel test af systemets kernefunktionalitet, da real-time samarbejde og synkronisering mellem flere klienter udgjorde en central del af projektets kompleksitet.

Testene blev primært udført gennem flere samtidige browserfaner og klienter for at verificere, at ændringer blev synkroniseret korrekt mellem brugere uden datakonflikter. Derudover blev persistence af dokumentdata testet ved genindlæsning af sider og genåbning af noter for at sikre, at dokumenttilstanden blev gemt korrekt i databasen.

Fokus for testarbejdet har været at validere systemets stabilitet, synkronisering og datahåndtering frem for automatiseret testning.

Særligt test af samtidige ændringer var vigtig for at verificere, at CRDT-baseret synkronisering fungerede korrekt under samarbejde mellem flere brugere.

Scenarie	Forventet	Udfald
Oprettelse af fag	Nyt fag oprettes korrekt	OK
Oprettelse af note	Note gemmes i databasen	OK
Navigation mellem fag og noter	Brugeren navigerer korrekt mellem sider	OK
Real-time redigering	Flere klienter ser ændringer samtidigt	OK
Persistence af noter	Dokumentet bevares efter reload	OK
Flere samtidige brugere	Konflikter håndteres korrekt	OK

Krav vs. leveret

De centrale krav til systemet er opfyldt. Brugere kan oprette og organisere noter gennem fag og emner, og systemet understøtter real-time samarbejde mellem flere klienter.

Derudover er persistence af dokumenttilstand implementeret, så noter gemmes og kan genskabes efter genindlæsning af systemet.

Enkelte områder er dog kun delvist implementeret. Dette gælder blandt andet avanceret rolle- og rettighedsstyring, automatiserede tests samt yderligere UI/UX forbedringer. Fokus har primært været på at implementere og stabilisere kernefunktionaliteten i systemet.

Kendte begrænsninger

Systemet indeholder fortsat enkelte begrænsninger og områder, der kan forbedre i fremtidige versioner.

Der er eksempelvis ikke implementeret avanceret rettighedsstyring mellem brugere, hvilket betyder, at samarbejdsfunktionaliteten fortsat er relativt simpel. Derudover mangler systemet versionshistorik og offline support.

Projektet indeholder heller ikke automatiserede tests, da fokus primært har været på implementering af kernefunktionalitet og real-time samarbejde.

Brugergrænsefladen kan ligeledes forbedres yderligere gennem mere avanceret design og bedre responsivitet på forskellige enheder.

REFLEKSION

Egen læring

Projektet har givet en dybere forståelse for udvikling af moderne webapplikationer og særligt arbejdet med real-time systemer. Under udviklingen er der opnået erfaring med både frontend, backend, databasedesign og WebSocket-baseret kommunikation.

En af de største læringer har været arbejdet med Yjs og Hocuspocus, hvor der blev opnået indsigt i håndtering af samtidige ændringer gennem CRDT-teknologi. Dette gav en større forståelse for kompleksiteten bag real-time samarbejde og synkronisering mellem klienter.

Derudover har projektet givet erfaring med arkitekturvalg i Next.js, herunder forskellen mellem server components og client components, samt hvordan datahåndtering og rendering påvirker applikationens struktur og performance.

Projektet har også styrket forståelsen for betydningen af modulær opbygning, versionsstyring og løbende refaktorering i takt med at projektets kompleksitet voksede.

Fremtidige forbedringer

1. Implementering af avanceret rolle- og rettighedsstyring
2. Versionshistorik for noter
3. Offline support
4. Bedre responsivt design og UI/UX
5. Containerisering og deployment med Docker

Afsluttende kommentarer

Projektet har resulteret i en funktionel platform til organisering og samarbejde om noter med understøttelse af real-time redigering mellem flere brugere.

Udviklingsprocessen har givet erfaring med både moderne webudvikling, databasedesign og real-time systemer, samtidig med at projektet har illustreret vigtigheden af struktur, arkitektur og løbende refaktorering i større applikationer.

Projektet har desuden givet indsigt i kompleksiteten bag samarbejdssystemer og hvordan moderne værktøjer som Yjs og Hocuspocus kan anvendes til at håndtere samtidige ændringer i real-time.